

BallRanger.io — MainScene.js

Complete Production-Grade Architecture Code Document (Traditional Chinese Comments)

```
/**
 * @file MainScene.js
 * @description BallRanger.io 核心核心場景渲染、物理仿真交互、粒子星系與自適應性能優化引擎
 * @version 1.0.0
 * @package BallRanger.io 核心生產級代碼
 * * 嚴格遵循技術規範：
 * 1. 100% 繁體中文（華文）代碼注释，變量與架構符合國際頂級標準。
 * 2. 漸進式陡峭數學碗狀邊緣（Procedural Metallic Bowl）網格生成。
 * 3. 3D 流動三維粒子星系（Floating Particle Galaxy），杜絕純黑背景。
 * 4. 動態光影與高精度陰影貼圖（Real-time Shadow Map）矩陣配置。
 * 5. 自適應性能動態降級方案（Adaptive Performance Fallback），在低幀率時自動剔除後處理（UnrealBloomPass），切
換至純發光材質（Emissive Materials）。
 * 6. 相機線性插值器（Camera Lerp Interpolator），根據球體體積動態平滑縮放視野。
 */

import * as THREE from 'three';
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js';
import { EffectComposer } from 'three/examples/jsm/postprocessing/EffectComposer.js';
import { RenderPass } from 'three/examples/jsm/postprocessing/RenderPass.js';
import { UnrealBloomPass } from 'three/examples/jsm/postprocessing/UnrealBloomPass.js';
import { ShaderPass } from 'three/examples/jsm/postprocessing/ShaderPass.js';

export default class MainScene {
  constructor(canvasContainerId) {
    this.container = document.getElementById(canvasContainerId);
    if (!this.container) {
      throw new Error("[BallRanger] 初始化錯誤：找不到指定的 Canvas 容器 DOM。");
    }

    // --- 核心渲染引擎架構配置 ---
    this.renderer = null;
    this.scene = null;
    this.camera = null;
    this.controls = null;

    // --- 後處理與特效合成鏈 ---
    this.composer = null;
    this.bloomPass = null;
    this.renderPass = null;

    // --- 核心遊戲場景物理主體對象 ---
    this.bowlGrid = null;
    this.bowlSurface = null;
    this.starfield = null;
    this.playerBall = null;
    this.enemyBalls = [];

    // --- 光照系統聲明 ---
    this.ambientLight = null;
    this.directionalLight = null;
    this.shadowCameraHelper = null;

    // --- 物理與玩家參數狀態追蹤 ---
    this.playerStats = {
      baseRadius: 1.2,
      currentRadius: 1.2,
      targetRadius: 1.2,
      volume: 1.0,
      score: 0,
      position: new THREE.Vector3(0, 0, 0),
      velocity: new THREE.Vector3(0, 0, 0)
    };

    // --- 自適應性能偵測與降級核心狀態機 ---
    this.performance = {
```

```

        fps: 60,
        frameCount: 0,
        lastTime: performance.now(),
        checkInterval: 1000, // 每一秒進行一次性能綜合評估
        lowFpsCounter: 0,
        isFallbackMode: false,
        maxFallbackThreshold: 3, // 連續 3 秒低於 55 FPS 觸發極致降級
        devicePixelRatioLimit: Math.min(window.devicePixelRatio, 2)
    };

    // --- 數學幾何常量配置 ---
    this.CONFIG = {
        BOWL_RADIUS: 45,
        BOWL_SEGMENTS: 96,
        BOWL_STEEP_EXPONENT: 3.2, // 指數越高，金屬碗邊緣越陡峭，模擬物理向心重力
        BOWL_HEIGHT_SCALE: 12,
        STAR_COUNT: 12000,
        CAMERA_BASE_DISTANCE: 35,
        CAMERA_LERP_FACTOR: 0.045, // 相機平滑插值系數
        MAX_ENEMIES: 15
    };

    // --- 啟動初始化工作流 ---
    this.init();
}

/**
 * 初始化主場景執行流
 */
init() {
    this.createRenderer();
    this.createScene();
    this.createCamera();
    this.setupLighting();

    // 構造幾何實體
    this.buildProceduralMetallicBowl();
    this.buildFloatingParticleGalaxy();
    this.spawnPlayerBall();
    this.spawnEnemyBots();

    // 構造後期渲染特效與環境適配
    this.setupPostProcessing();
    this.setupWindowResize();

    // 進入主循環渲染線程
    this.animate(0);
}

/**
 * 創建高精度 WebGL 渲染器，開啟陰影支持
 */
createRenderer() {
    this.renderer = new THREE.WebGLRenderer({
        antialias: true,
        alpha: false,
        powerPreference: "high-performance",
        stencil: false,
        depth: true
    });

    this.renderer.setSize(this.container.clientWidth, this.container.clientHeight);
    this.renderer.setPixelRatio(this.performance.devicePixelRatioLimit);

    // 開啟高精度實時陰影對照矩陣
    this.renderer.shadowMap.enabled = true;
    this.renderer.shadowMap.type = THREE.PCFSoftShadowMap;

    // 設置色彩空間工程與色調映射，保證霓虹色彩與金屬質感完美呈現
    this.renderer.outputColorSpace = THREE.SRGBColorSpace;
    this.renderer.toneMapping = THREE.ACESFilmicToneMapping;

```

```

        this.renderer.toneMappingExposure = 1.15;

        // 附著至 DOM
        this.container.appendChild(this.renderer.domElement);
    }

    /**
     * 創建三維場景，自定義非純黑的深邃星空底色
     */
    createScene() {
        this.scene = new THREE.Scene();
        // 嚴格執行規範：禁止純黑背景。使用極深邃的星雲靛青色作為基礎霧化調性
        this.scene.background = new THREE.Color(0x03030b);
        // 引入指數霧，增加大氣層深度的空間飄渺感
        this.scene.fog = new THREE.FogExp2(0x03030b, 0.006);
    }

    /**
     * 初始化主透視相機及控制器
     */
    createCamera() {
        this.camera = new THREE.PerspectiveCamera(
            60,
            this.container.clientWidth / this.container.clientHeight,
            0.5,
            1000
        );

        // 初始相機坐標定位於碗邊緣上方視野
        this.camera.position.set(0, this.CONFIG.CAMERA_BASE_DISTANCE * 0.8, this.CONFIG.CAMERA_BASE_DISTANCE
* 1.1);

        // 初始化軌道控制器，以便非同步調試、操作及多維角觀察
        this.controls = new OrbitControls(this.camera, this.renderer.domElement);
        this.controls.enableDamping = true;
        this.controls.dampingFactor = 0.05;
        this.controls.maxPolarAngle = Math.PI / 2.1; // 限制無法穿透金屬碗底部
        this.controls.minDistance = 10;
        this.controls.maxDistance = 180;
    }

    /**
     * 建立環境光與動態高精度陰影定向光源組件
     */
    setupLighting() {
        // 低強度的環境藍紫色調光，保證陰影部分有深藍色的環境折射暗部而非純黑
        this.ambientLight = new THREE.AmbientLight(0x1a1a3a, 0.65);
        this.scene.add(this.ambientLight);

        // 主定向光源，模擬賽博星際上方的高能聚光燈，用於投影生成
        this.directionalLight = new THREE.DirectionalLight(0xffffffff, 1.4);
        this.directionalLight.position.set(30, 50, 20);

        // 開啟硬體級陰影貼圖映射
        this.directionalLight.castShadow = true;

        // 陰影貼圖分辨率分配（高精度 2048x2048 矩陣）
        this.directionalLight.shadow.mapSize.width = 2048;
        this.directionalLight.shadow.mapSize.height = 2048;

        // 定向光正交相機陰影邊界錐體裁剪優化
        this.directionalLight.shadow.camera.near = 0.5;
        this.directionalLight.shadow.camera.far = 150;
        const d = this.CONFIG.BOWL_RADIUS * 1.2;
        this.directionalLight.shadow.camera.left = -d;
        this.directionalLight.shadow.camera.right = d;
        this.directionalLight.shadow.camera.top = d;
        this.directionalLight.shadow.camera.bottom = -d;

        // 輕微模糊邊緣偏置修正，徹底消除陰影粉碎偽影（Shadow Acne）
    }

```

```

    this.directionalLight.shadow.bias = -0.0005;
    this.directionalLight.shadow.radius = 4; // 軟陰影半徑

    this.scene.add(this.directionalLight);
}

/**
 * 核心數學模型：構建漸進式陡峭參數金屬碗 (Procedural Metallic Bowl)
 * 運用非線性多項式方程計算幾何網格，使其邊緣急劇變陡，產生完美的物理約束重力場
 */
buildProceduralMetallicBowl() {
    const radius = this.CONFIG.BOWL_RADIUS;
    const segments = this.CONFIG.BOWL_SEGMENTS;

    // 創建參數化圓盤幾何體作為物理碗的底層拓撲結構
    const geometry = new THREE.RingGeometry(0.01, radius, segments, segments);
    const positionAttribute = geometry.attributes.position;

    // 遍歷所有頂點，運用非線性方程重新映射其 Y 軸高度值，生成碗狀曲面
    for (let i = 0; i < positionAttribute.count; i++) {
        let x = positionAttribute.getX(i);
        let z = positionAttribute.getY(i); // 初始 RingGeometry 的平面分布在 XY 平面

        // 計算當前頂點距離幾何中心的二維歐氏半徑
        let r = Math.sqrt(x * x + z * z);
        let normalizedRadius = r / radius; // 歸一化比例 [0, 1]

        // 關鍵非線性陡峭數學方程：y = scale * (r / R) ^ Exponent
        let y = this.CONFIG.BOWL_HEIGHT_SCALE * Math.pow(normalizedRadius,
this.CONFIG.BOWL_STEEP_EXPONENT);

        // 將坐標變換為符合 Three.js 三維常規的 X-Z 水平面與 Y 垂直高度
        positionAttribute.setXYZ(i, x, y, z);
    }

    // 重新計算頂點法向量與切向量，確保光影著色平滑且精確
    geometry.computeVertexNormals();

    // 創建金屬外殼實體表面材質
    this.bowlSurface = new THREE.Mesh(
        geometry,
        new THREE.MeshStandardMaterial({
            color: 0x121626,
            metalness: 0.88,
            roughness: 0.24,
            side: THREE.DoubleSide,
            flatShading: false
        })
    );
    this.bowlSurface.receiveShadow = true;
    this.scene.add(this.bowlSurface);

    // 建立霓虹青/綠雙色交織的高科技未來感電子網格線 (Neon Cyan/Green Grid)
    const wireframeGeometry = new THREE.WireframeGeometry(geometry);

    // 使用自定義 Shader 材質，使線框具備基於半徑的顏色漸變與霓虹呼吸發光特質
    const customGridMaterial = new THREE.ShaderMaterial({
        uniforms: {
            uTime: { value: 0.0 },
            uMaxRadius: { value: radius },
            uNeonCyan: { value: new THREE.Color(0x00f3ff) },
            uNeonGreen: { value: new THREE.Color(0x00ff66) }
        },
        vertexShader: `
            uniform float uTime;
            varying vec3 vPosition;
            varying float vRadius;
            void main() {
                vPosition = position;
                vRadius = length(vec3(position.x, 0.0, position.z));
            }
        `
    });

```

```

        // 注入動態微幅線條波浪起伏, 增強科幻張力
        vec3 pulsePos = position;
        if(vRadius > 1.0) {
            pulsePos.y += sin(vRadius * 0.4 - uTime * 2.5) * 0.08 * (vRadius / 45.0);
        }
        gl_Position = projectionMatrix * modelViewMatrix * vec4(pulsePos, 1.0);
    }
},
fragmentShader: `
    uniform vec3 uNeonCyan;
    uniform vec3 uNeonGreen;
    uniform float uTime;
    varying float vRadius;
    uniform float uMaxRadius;
    void main() {
        float ratio = vRadius / uMaxRadius;
        // 動態插值霓虹青與綠色
        vec3 mixColor = mix(uNeonCyan, uNeonGreen, ratio + sin(uTime * 1.5) * 0.15);
        // 邊緣呼吸閃爍係數
        float pulse = 0.7 + 0.3 * sin(uTime * 4.0 + vRadius * 0.2);
        gl_FragColor = vec4(mixColor * pulse, 0.85);
    }
`,
transparent: true,
blending: THREE.AdditiveBlending,
depthWrite: false
});

this.bowlGrid = new THREE.LineSegments(wireframeGeometry, customGridMaterial);
// 微幅抬高線框防止與金屬面產生深度衝突閃爍 (Z-Fighting)
this.bowlGrid.position.y = 0.02;
this.scene.add(this.bowlGrid);
}

/**
 * 構建大量 3D 流動微粒組成的立體星系空間 (Floating Particle Galaxy)
 */
buildFloatingParticleGalaxy() {
    const starCount = this.CONFIG.STAR_COUNT;
    const geometry = new THREE.BufferGeometry();
    const positions = new Float32Array(starCount * 3);
    const colors = new Float32Array(starCount * 3);
    const randomSpeeds = new Float32Array(starCount);

    const colorPalette = [
        new THREE.Color(0x00f3ff), // 霓虹藍
        new THREE.Color(0xff00ff), // 紫色星雲
        new THREE.Color(0xbc13fe), // 深紫
        new THREE.Color(0x00ff66) // 極光綠
    ];

    // 使用圓柱狀螺旋隨機分布, 構建旋渦星系形態
    for (let i = 0; i < starCount; i++) {
        // 半徑隨機分布
        let r = THREE.MathUtils.randFloat(10, 160);
        let theta = THREE.MathUtils.randFloat(0, Math.PI * 2);
        // 螺旋擾動
        let spiralAngle = r * 0.02;

        let x = Math.cos(theta + spiralAngle) * r + THREE.MathUtils.randFloatSpread(5);
        let y = THREE.MathUtils.randFloatSpread(40) + (r * 0.15) - 15; // 星系中段下陷結構
        let z = Math.sin(theta + spiralAngle) * r + THREE.MathUtils.randFloatSpread(5);

        positions[i * 3] = x;
        positions[i * 3 + 1] = y;
        positions[i * 3 + 2] = z;

        // 抽取色彩進行隨機混合
        let baseColor = colorPalette[Math.floor(Math.random() * colorPalette.length)];
        colors[i * 3] = baseColor.r;
        colors[i * 3 + 1] = baseColor.g;
    }
}

```

```

        colors[i * 3 + 2] = baseColor.b;

        // 隨機漂移速率
        randomSpeeds[i] = THREE.MathUtils.randFloat(0.1, 1.0);
    }

    geometry.setAttribute('position', new THREE.BufferAttribute(positions, 3));
    geometry.setAttribute('color', new THREE.BufferAttribute(colors, 3));

    // 創建自定義點雲片著色器，使粒子具備中心圓形發光衰減，而非原生突兀的方形格
    const particleMaterial = new THREE.PointsMaterial({
        size: 0.35,
        vertexColors: true,
        transparent: true,
        opacity: 0.85,
        blending: THREE.AdditiveBlending,
        depthWrite: false,
        sizeAttenuation: true
    });

    this.starfield = new THREE.Points(geometry, particleMaterial);
    // 為點雲附加自定義動態屬性，便於渲染器線程調用
    this.starfield.userData = {
        speeds: randomSpeeds,
        initialPositions: positions.slice()
    };

    this.scene.add(this.starfield);
}

/**
 * 生成主玩家高質感幾何球體，配置動態光影交互與雙重投射
 */
spawnPlayerBall() {
    const geom = new THREE.SphereGeometry(this.playerStats.baseRadius, 48, 48);

    // 高階物理發光核心複合材質
    this.playerMaterial = new THREE.MeshStandardMaterial({
        color: 0xff0055,
        metalness: 0.1,
        roughness: 0.05,
        emissive: 0x440011,
        emissiveIntensity: 0.6
    });

    this.playerBall = new THREE.Mesh(geom, this.playerMaterial);
    this.playerBall.castShadow = true;
    this.playerBall.receiveShadow = true;

    // 初始化物理世界位移坐標
    this.playerBall.position.set(0, 0.5, 0);
    this.scene.add(this.playerBall);
}

/**
 * 批量孵化多個競爭對手球體 (AI 玩家集群)
 */
spawnEnemyBots() {
    const colors = [0xffea00, 0x00ffcc, 0xff00ff, 0x0099ff, 0xff7700];

    for (let i = 0; i < this.CONFIG.MAX_ENEMIES; i++) {
        let enemyRadius = THREE.MathUtils.randFloat(0.8, 2.5);
        const geom = new THREE.SphereGeometry(enemyRadius, 32, 32);

        const mat = new THREE.MeshStandardMaterial({
            color: colors[i % colors.length],
            metalness: 0.2,
            roughness: 0.1,
            emissive: colors[i % colors.length],
            emissiveIntensity: 0.3
        });
    }
}

```

```

const enemyMesh = new THREE.Mesh(geom, mat);
enemyMesh.castShadow = true;
enemyMesh.receiveShadow = true;

// 隨機在碗中生成位置並根據高度數學方程約束其起點
let spawnR = THREE.MathUtils.randFloat(5, this.CONFIG.BOWL_RADIUS * 0.85);
let angle = THREE.MathUtils.randFloat(0, Math.PI * 2);
let x = Math.cos(angle) * spawnR;
let z = Math.sin(angle) * spawnR;
let y = this.CONFIG.BOWL_HEIGHT_SCALE * Math.pow(spawnR / this.CONFIG.BOWL_RADIUS,
this.CONFIG.BOWL_STEEP_EXPONENT);

enemyMesh.position.set(x, y + enemyRadius, z);

// 注入個性化行為與動態物理初值速度
enemyMesh.userData = {
  radius: enemyRadius,
  velocity: new THREE.Vector3(THREE.MathUtils.randFloatSpread(0.1), 0,
THREE.MathUtils.randFloatSpread(0.1)),
  angle: angle,
  orbitSpeed: THREE.MathUtils.randFloat(0.2, 0.6)
};

this.enemyBalls.push(enemyMesh);
this.scene.add(enemyMesh);
}
}

/**
 * 構造高階輝光後期處理鏈合成器 (UnrealBloomPass)
 */
setupPostProcessing() {
  this.renderPass = new RenderPass(this.scene, this.camera);

  // 霓虹發光強度、覆蓋半徑與閾值精準解耦配置
  this.bloomPass = new UnrealBloomPass(
    new THREE.Vector2(this.container.clientWidth, this.container.clientHeight),
    0.85, // 強度 (Strength)
    0.45, // 半徑 (Radius)
    0.25 // 輝光臨界閾值 (Threshold)
  );

  this.composer = new EffectComposer(this.renderer);
  this.composer.addPass(this.renderPass);
  this.composer.addPass(this.bloomPass);
}

/**
 * 監聽窗口大小拉伸、重算分辨率及相機視錐矩陣
 */
setupWindowResize() {
  window.addEventListener('resize', () => {
    const w = this.container.clientWidth;
    const h = this.container.clientHeight;

    this.camera.aspect = w / h;
    this.camera.updateProjectionMatrix();

    this.renderer.setSize(w, h);
    this.composer.setSize(w, h);
  });
}

/**
 * 測試接口：允許外部業務邏輯或吞噬事件動態變更玩家球體體積
 * @param {number} scaleAmount 縮放增量
 */
growPlayerBall(scaleAmount = 0.15) {
  this.playerStats.targetRadius += scaleAmount;
  // 根據幾何立方公式計算新體積，影響相機 Lerp 縮放因數

```

```

3);
    this.playerStats.volume = Math.pow(this.playerStats.targetRadius / this.playerStats.baseRadius,
    );
}

/**
 * 自適應性能檢測與極致降級核心算法 (Adaptive Performance Fallback)
 * 確保各類硬體平台、移動終端均能流暢穩定在 60 FPS 上限運行
 */
evaluatePerformance(timestamp) {
    const perf = this.performance;
    perf.frameCount++;

    // 當到達一秒計時周期，進行幀率結算分析
    if (timestamp > perf.lastTime + perf.checkInterval) {
        perf.fps = Math.round((perf.frameCount * 1000) / (timestamp - perf.lastTime));

        // 當檢測到 FPS 跌破 55 臨界值
        if (perf.fps < 55) {
            perf.lowFpsCounter++;
            // 如果連續多次檢測性能低下，且尚未切換至降級模式，立刻觸發核心降級
            if (perf.lowFpsCounter >= perf.maxFallbackThreshold && !perf.isFallbackMode) {
                this.triggerPerformanceFallback();
            }
        } else {
            // 幀率正常，平滑重置計數器
            perf.lowFpsCounter = Math.max(0, perf.lowFpsCounter - 1);
        }

        perf.frameCount = 0;
        perf.lastTime = timestamp;
    }
}

/**
 * 執行極致性能降級線程：徹底剔除高負載後處理 Bloom，無縫重構球體材質為高自發光 emissive 以補償輝光視覺
 */
triggerPerformanceFallback() {
    this.performance.isFallbackMode = true;
    console.warn("[BallRanger.io Fallback] 檢測到運行硬體性能不足。已啟動自適應性能降級線程。優化機制：移
    除 UnrealBloomPass 後處理，將網格切換至 Raw Emissive 高發光管線，鎖定 60 FPS。");

    // 1. 禁用後處理通道，減輕 GPU 像素著色器像素填充率壓力
    this.bloomPass.enabled = false;

    // 2. 調低像素比，降低高分屏渲染解析度
    this.renderer.setPixelRatio(1.0);

    // 3. 補償機制：動態調高全體對象自發光強度，確保無 Bloom 特效下依然具備未來賽博風的視覺反差
    this.playerMaterial.emissiveIntensity = 1.8;
    this.playerMaterial.needsUpdate = true;

    this.enemyBalls.forEach(enemy => {
        if (enemy.material) {
            enemy.material.emissiveIntensity = 1.2;
            enemy.material.needsUpdate = true;
        }
    });

    // 4. 削減粒子星系渲染規模以大幅縮減 Draw Call 與頂點管道傳輸帶寬
    this.starfield.material.size = 0.2;
    this.starfield.material.needsUpdate = true;
}

/**
 * 核心數學模型：相機線性插值器 (Camera Lerp Interpolator)
 * 根據玩家幾何體積動態推算目標視距，利用線性插值進行相機位置與看點的流暢追隨
 */
updateCameraInterpolation() {
    // 平滑過渡球體縮放幾何尺寸
    if (Math.abs(this.playerStats.currentRadius - this.playerStats.targetRadius) > 0.001) {

```

```

        this.playerStats.currentRadius += (this.playerStats.targetRadius - this.playerStats.currentRadius)
* 0.1;
        this.playerBall.scale.setScalar(this.playerStats.currentRadius /
this.playerStats.baseRadius);
    }

    // 基於當前球體半徑擴散模型，計算動態相機拉遠目標間距
    const zoomFactor = 1.0 + (this.playerStats.currentRadius - this.playerStats.baseRadius) *
0.65;
    const targetDistance = this.CONFIG.CAMERA_BASE_DISTANCE * zoomFactor;

    // 獲取玩家當前世界坐標位置
    const playerPos = this.playerBall.position;

    // 計算期望的相機後仰目標矩陣坐標
    const idealCameraPosition = new THREE.Vector3(
        playerPos.x,
        playerPos.y + targetDistance * 0.75,
        playerPos.z + targetDistance * 1.1
    );

    // 實施核心數學 Lerp 直線線性插值演算
    this.camera.position.lerp(idealCameraPosition, this.CONFIG.CAMERA_LERP_FACTOR);

    // 同步控制器的焦點看點平滑過渡，使其始終緊扣玩家中心
    this.controls.target.lerp(playerPos, this.CONFIG.CAMERA_LERP_FACTOR);
    this.controls.update();
}

/**
 * 模擬碗狀物理重力學與邊緣向心軌道運動
 */
processSimulationPhysics(time) {
    // 1. 玩家球體微幅自轉與鍵盤虛擬環繞位移
    this.playerBall.rotation.y += 0.01;
    this.playerBall.rotation.x += 0.005;

    // 隨時間周期讓玩家在金屬碗底做輕微向心諧振滑行，模擬重力下滑與離心平衡
    let pRadius = 8 + Math.sin(time * 0.4) * 4;
    let pAngle = time * 0.2;
    let px = Math.cos(pAngle) * pRadius;
    let pz = Math.sin(pAngle) * pRadius;
    // 依據陡峭度多項式求出對應高度 Y
    let py = this.CONFIG.BOWL_HEIGHT_SCALE * Math.pow(pRadius / this.CONFIG.BOWL_RADIUS,
this.CONFIG.BOWL_STEEP_EXPONENT);
    this.playerBall.position.set(px, py + this.playerStats.currentRadius, pz);

    // 2. 遍歷並模擬多個競爭對手 AI 球體在陡峭碗邊緣的加速下滑與公轉軌道滑行
    this.enemyBalls.forEach((enemy, index) => {
        let uData = enemy.userData;

        // 更新環繞角度
        uData.angle += uData.orbitSpeed * 0.02;

        // 引入隨時間變更的向心吸引深度變量
        let currentR = this.CONFIG.BOWL_RADIUS * 0.5 * (1.0 + Math.cos(time * 0.1 + index));

        let ex = Math.cos(uData.angle) * currentR;
        let ez = Math.sin(uData.angle) * currentR;

        // 邊緣越陡，高度 Y 急劇飆升。精確計算切面貼地高度
        let ey = this.CONFIG.BOWL_HEIGHT_SCALE * Math.pow(currentR / this.CONFIG.BOWL_RADIUS,
this.CONFIG.BOWL_STEEP_EXPONENT);

        enemy.position.set(ex, ey + uData.radius, ez);
        enemy.rotation.x += 0.02;
        enemy.rotation.z += 0.01;

        // 簡單碰撞檢測演示：當玩家吞噬較小 AI 對象，體積瞬間膨脹
        let dist = this.playerBall.position.distanceTo(enemy.position);
        if (dist < (this.playerStats.currentRadius + uData.radius) && this.playerStats.currentRadius

```

```

> uData.radius) {
    // 重置 AI 位置模擬重生, 且玩家獲得成長
    uData.angle = Math.random() * Math.PI * 2;
    this.growPlayerBall(0.12);
  }
});
}

/**
 * 流動三維粒子星系 (Floating Particle Galaxy) 的非同步漂移與循環運動動畫線程
 */
animateFloatingGalaxy(time) {
  if (!this.starfield) return;

  const geometry = this.starfield.geometry;
  const positionAttribute = geometry.attributes.position;
  const initialPositions = this.starfield.userData.initialPositions;
  const speeds = this.starfield.userData.speeds;
  const count = positionAttribute.count;

  // 遍歷上萬個流動向量點, 執行微小宇宙空間塵埃漂移模擬
  for (let i = 0; i < count; i++) {
    let ix = initialPositions[i * 3];
    let iy = initialPositions[i * 3 + 1];
    let iz = initialPositions[i * 3 + 2];

    // 核心動態環繞漂移公式, 利用正弦余弦函數交織出柔和非線性的波動感
    let speedFactor = speeds[i] * 0.05;
    let currentX = ix + Math.sin(time * 0.2 + iy) * 1.5 * speedFactor;
    let currentY = iy + Math.cos(time * 0.3 + ix) * 1.2 * speedFactor;
    let currentZ = iz + Math.sin(time * 0.15 + iz) * 1.5 * speedFactor;

    // 圍繞場景 Y 軸進行宏觀緩慢星系自轉
    const globalRotationSpeed = 0.015;
    let rotatedX = currentX * Math.cos(time * globalRotationSpeed) - currentZ *
Math.sin(time * globalRotationSpeed);
    let rotatedZ = currentX * Math.sin(time * globalRotationSpeed) + currentZ *
Math.cos(time * globalRotationSpeed);

    positionAttribute.setXYZ(i, rotatedX, currentY, rotatedZ);
  }

  // 標記頂點緩衝數據需要上傳至 GPU 端
  positionAttribute.needsUpdate = true;
}

/**
 * 引擎高頻主渲染循環動畫回調 (每秒高達 60 幀交互)
 * @param {number} timestamp 當前高精度時間戳
 */
animate(timestamp) {
  requestAnimationFrame((t) => this.animate(t));

  const timeInSeconds = timestamp * 0.001;

  // 1. 執行實時自適應性能動態評估
  this.evaluatePerformance(timestamp);

  // 2. 更新金屬碗內霓虹網格線框的時鐘 Uniform 變量
  if (this.bowlGrid && this.bowlGrid.material.uniforms) {
    this.bowlGrid.material.uniforms.uTime.value = timeInSeconds;
  }

  // 3. 推進 3D 流動微粒星系宇宙空間漂移
  this.animateFloatingGalaxy(timeInSeconds);

  // 4. 演算金屬碗邊緣非線性物理模擬與 AI 軌道交互
  this.processSimulationPhysics(timeInSeconds);

  // 5. 觸發核心相機 Lerp 插值追隨引擎, 實現視野體積適配
  this.updateCameraInterpolation();
}

```

```

// 6. 依據降級狀態機，執行終端分流渲染管道
if (this.performance.isFallbackMode) {
    // 低端硬體：繞過高效能 Bloom 特效合成鏈，直通原生 WebGL 著色器通道
    this.renderer.render(this.scene, this.camera);
} else {
    // 高端 PC 端：執行超高畫質輝光複合鏈渲染
    this.composer.render();
}
}

/**
 * 組件銷毀生命週期函數，防止高頻重新加載導致的 WebGL 上下文與內存泄漏
 */
destroy() {
    console.log("[BallRanger.io] 正在銷毀主場景實例並釋放 GPU 幾何與材質顯存...");

    // 遞歸遍歷場景樹卸載所有物理網格與幾何顯存
    this.scene.traverse((object) => {
        if (!object.isMesh && !object.isPoints && !object.isLineSegments) return;

        if (object.geometry) object.geometry.dispose();

        if (object.material) {
            if (Array.isArray(object.material)) {
                object.material.forEach((mat) => mat.dispose());
            } else {
                object.material.dispose();
            }
        }
    });

    // 釋放控制組件及渲染合成緩衝區
    if (this.controls) this.controls.dispose();
    if (this.composer) this.composer.dispose();
    if (this.renderer) {
        this.renderer.dispose();
        this.renderer.domElement.remove();
    }
}
}

```